UNIVERSIDAD DE SAN CARLOS DE GUATEMALA

CENTRO UNIVERSITARIO DE OCCIDENTE

CARRERA DE INGENIERÍA EN CIENCIAS Y SISTEMAS

SISTEMAS DE BASES DE DATOS 2

Ingeniero Daniel González

Segundo Semestre de 2026

MANUAL TÉCNICO

Guía de instalación, arquitectura, configuración, modelo de datos y mantenimiento

USAC Analytics — Proyecto 2

ESTUDIANTE

Byron Fernando Torres Ajxup

Carné: 201731523

Quetzaltenango, octubre de 2026

Propósito general y arquitectura

El presente Manual Técnico describe de manera integral la instalación, configuración, arquitectura interna, modelo de datos y procedimientos de mantenimiento del sistema **USAC Analytics**. La plataforma está orientada al análisis analítico, relacional e híbrido del histórico de evaluaciones de Pruebas Específicas de la Universidad de San Carlos de Guatemala, permitiendo explorar volúmenes masivos de datos a través de una interfaz interactiva de Dashboard, motor de consultas analíticas, explorador visual de grafos e importador masivo de datos.

El sistema adopta una arquitectura desacoplada organizada en tres capas principales:

- **Capa de Presentación (Frontend):** Desarrollada en React 18 con TypeScript y Tailwind CSS, ofreciendo una experienciaSPA interactiva con paneles estadísticos, filtros dinámicos, tablas paginadas y renderizado de grafos mediante Cytoscape.js.
- **Capa de Servicios y Negocio (Backend):** Construida sobre Python 3.12 y FastAPI, encargada de la exposición de la API RESTFUL, lógica ETL, normalización de datos, validaciones de integridad y orquestación de consultas hacia persistencia.

Capa de Persistencia Híbrida: Implementa una arquitectura Políglota de Persistencia compuesta por MongoDB para el almacenamiento documental orientado a agregaciones, e instancias de Neo4j para el análisis relacional y consultas analíticas mediante Cypher.

Organización del repositorio

La estructura interna del proyecto está estructurada modularmente para asegurar una estricta separación de responsabilidades entre el backend, frontend y los scripts de base de datos:

```
None
PROYECTO2_BD2/
├── backend/
│   ├── app/
│   │   ├── api/            # Endpoints REST (health, estadísticas, grafos,
búsqueda, importación)
│   │   ├── core/           # Configuración de variables de entorno y conexión
a bases de datos
│   │   ├── db/             # Clientes de MongoDB y Neo4j
│   │   ├── etl/            # Transformación (transform.py), carga MongoDB y
Neo4j
│   │   ├── services/       # Lógica analítica de MongoDB (20 consultas) y
Cypher (5 consultas)
│   │   └── main.py         # Punto de entrada de la aplicación FastAPI y CORS
│   ├── .env.example        # Plantilla de configuración de entorno
│   └── requirements.txt    # Dependencias de Python
├── frontend/
```

```
|   ├── src/
|   |   ├── components/      # Componentes UI reutilizables y gráficos
|   |   ├── pages/           # Dashboard, Consultas, Explorador de Grafos,
Búsqueda, Importación
|   |   └── services/        # Cliente HTTP Axios para llamadas a la API
|   ├── package.json         # Dependencias de Node.js
|   └── vite.config.ts       # Configuración de Vite
└── database/
    ├── mongodb/             # Scripts de inicialización e índices
    └── neo4j/               # Consultas Cypher y restricciones de unicidad
```

Requisitos y configuración del entorno

Para la correcta ejecución del sistema, el entorno de despliegue debe contar con las siguientes tecnologías instaladas y configuradas:

Requisitos de Software:

- **Python:** Versión 3.12 o superior.
- **Node.js y npm:** Node v18+ y npm v9+.
- **MongoDB:** Instancia ejecutable en `localhost:27017`.
- **Neo4j:** Servidor de grafos ejecutable en `neo4j://127.0.0.1:7687`.

Ejemplo del archivo de variables de entorno (`backend/.env`):

```
None
MONGO_URI=mongodb://localhost:27017
MONGO_DB=usac_pruebas_especificas
NEO4J_URI=neo4j://127.0.0.1:7687
NEO4J_USER=neo4j
NEO4J_PASSWORD=COLOCAR_CLAVE_LOCAL
NEO4J_DATABASE=neo4j
```

Nota importante: El archivo `.env` almacena credenciales privadas y sensibles, por lo que debe permanecer excluido del control de versiones mediante el archivo `.gitignore`.

Guía de ejecución del sistema

Ejecución del Backend (FastAPI)

Para inicializar el backend y los servicios de API REST, ejecute la siguiente secuencia de comandos desde la raíz del proyecto:

```shell
python -m venv backend/.venv
# En Linux/macOS: source backend/.venv/bin/activate
# En Windows: backend\.venv\Scripts\activate
pip install -r backend/requirements.txt
uvicorn app.main:app --reload --app-dir backend
```

Una vez iniciado, verifique la accesibilidad de la documentación interactiva Swagger en `http://127.0.0.1:8000/docs` y compruebe el diagnóstico de conexión en `/api/health`.

Ejecución del Frontend (React + Vite)

Para iniciar el servidor de desarrollo del cliente web:

```shell
cd frontend
npm install
npm run dev
```

La interfaz web estará disponible en `http://localhost:5173`. Para validar la compilación de producción, ejecute `npm run build`. Asegúrese de que el backend mantenga configurado el middleware de CORS para admitir el origen del cliente web.

Origen de datos y Proceso ETL

El flujo ETL (Extract, Transform, Load) comprende el procesamiento masivo de archivos CSV con el historial de evaluaciones. La canalización se compone de tres fases automatizadas:

- **Transformación y Normalización (**`backend/app/etl/transform.py`**):** Limpia campos obligatorios, estandariza formatos de fecha ISO, infiere el año de ingreso, calcula número de intento y deriva la condición de aprobación/reprobación.
- **Carga en MongoDB (**`backend/app/etl/load_mongodb.py`**):** Realiza inserciones por lotes optimizados mediante operaciones `upsert` tomando como clave de unicidad `id_registro`.
- **Sincronización en Neo4j (**`backend/app/etl/load_neo4j.py`**):** Efectúa la ingesta masiva en la base de grafos mediante declaraciones Cypher `MERGE`, garantizando la reutilización de nodos y evitando duplicados.

El endpoint `POST /api/importacion/csv` admite hasta 5 archivos simultáneos de 20 MB cada uno enviados vía `FormData`. La importación es tolerante a fallos: las filas con inconsistencias se registran en la colección `errores_carga` sin interrumpir el procesamiento del resto del lote.

Modelo Documental en MongoDB

MongoDB almacena la base documental principal optimizada para agregaciones analíticas complejas y búsquedas paginadas. El modelo consta de cuatro colecciones principales:

- **evaluaciones:** Almacena los registros individuales de pruebas. Colección indexada por `id_registro`, `correlativo`, `materia`, `carrera` y fechas.
- **historial_cargas:** Registra el métrico de ejecución de archivos procesados, incluyendo nombre, fecha, total de registros, aceptados y rechazados.
- **errores_carga:** Guarda el detalle técnico y motivo de rechazo de las filas no válidas durante el proceso de ingesta.

resumen_carrera: Colección auxiliar generada bajo demanda (vía `POST /api/estadisticas/generar-resumen-carrera`) que precalcula los consolidados por carrera para agilizar la respuesta de tableros de control.

Modelo de Grafos en Neo4j

Neo4j modela el dominio del sistema mediante una red de entidades interconectadas, optimizada para traversals profundos y relaciones complejas:

- **Etiquetas de Nodos:** `Aspirante`, `Evaluacion`, `Materia`, `Carrera`, `InstitucionTipo`, `Municipio`, `Departamento` y `PeriodoIngreso`.
- **Relaciones de Dominio:** `(:Aspirante)-[:REALIZO]->(:Evaluacion)`, `(:Evaluacion)-[:EVALUA]->(:Materia)`, `(:Evaluacion)-[:PARA_CARRERA]->(:Carrera)`, `(:Aspirante)-[:PROVIENE_DE_TIPO]->(:InstitucionTipo)`, `(:Aspirante)-[:PROCEDE_DE]->(:Municipio)`, `(:Municipio)-[:PERTENECE_A]->(:Departamento)` y `(:Evaluacion)-[:EN_PERIODO]->(:PeriodoIngreso)`.

Para mantener el rendimiento y la consistencia, la base de datos define índices y restricciones de unicidad sobre los identificadores principales (p. ej., `correlativo` en `Aspirante` e `id_registro` en `Evaluacion`).

Servicios de la API REST y Manejo de Errores

La API expone sus servicios mediante los siguientes grupos de endpoints catalogados:

- **Diagnóstico:** `GET /api/health`, `GET /api/health/mongodb`, `GET /api/health/neo4j` (monitoreo de conectividad).
- **Estadísticas MongoDB:** `GET /api/estadisticas/...` (20 consultas analíticas parametrizadas).
- **Consultas Cypher:** `GET /api/grafos/aprobaciones-materia`, `/aspirantes-departamento`, `/historial/{correlativo}`, `/carreras-departamento`, `/desempeno-intentos`.

- **Visualización de Grafos:** `GET /api/grafos/visualizar/{correlativo}` (devuelve nodos y aristas estructurados para Cytoscape).
- **Búsqueda Paginada:** `GET /api/busqueda/evaluaciones` (soporta filtros opcionales por materia, carrera, resultado y paginación).
- **Importación:** `POST /api/importacion/csv` (procesamiento masivo multipart).

El backend implementa un manejador estandarizado de excepciones utilizando respuestas HTTP con códigos de estado adecuados (400 Bad Request, 404 Not Found, 422 Unprocessable Entity, 500 Internal Server Error) acompañados de cargas JSON con mensajes descriptivos.

Consultas Analíticas MongoDB y Cypher

El servicio analítico de MongoDB (`backend/app/services/estadisticas.py`) implementa 20 tuberías de agregación (`aggregation pipelines`) utilizando operadores como `$match`, `$group`, `$sort` y `$project`. Distingue rigurosamente la cantidad de evaluaciones registradas frente al recuento de aspirantes únicos.

Las 5 consultas avanzadas de Neo4j (`backend/app/services/grafos.py`) ejecutan algoritmos sobre la red de grafos para determinar distribución territorial, históricos individuales, rendimiento por materia y correlación por tipo de institución.

Verificación de Integridad, Limitaciones y Pruebas

Verificación de Integridad:

Tras procesar el conjunto original de diez archivos CSV, la base de datos debe validar la paridad exacta registrando un total idéntico de **208,453 documentos en MongoDB** y **208,453 nodos Evaluacion en Neo4j**.

Pruebas de Compilación:

- **Backend:** `python -m compileall -q backend/app` (verifica sintaxis y compilación sin errores).
- **Frontend:** `cd frontend && npm run build` (comprueba la compilación de TypeScript y empaquetado producción).

Limitaciones y Posibles Mejoras:

Al tratarse de dos motores no relacionales desacoplados, la sincronización entre MongoDB y Neo4j no es transaccional en dos fases (2PC). Si ocurre una falla de red durante la segunda fase de ingesta, se requiere una ejecución de conciliación. Como mejoras futuras se contempla integrar colas de mensajes (RabbitMQ/Kafka) para sincronización asíncrona y capas de autenticación JWT para entornos de producción.